

1. Django Project Setup

Problem:

Write a command to create a new Django project named school_mgmt and start a new app named students.

Input Format:

Project name and App name

Output Format:

Django project and app should be created.

Example 1:

Input:

school_mgmt students

Output:

Project 'school_mgmt' created

App 'students' created

Answer:

```
django-admin startproject school_mgmt
```

```
cd school_mgmt
```

```
python manage.py startapp students
```

2. Django Model Creation

Problem:

Create a Django model Student with fields: name (CharField), age (IntegerField), email (EmailField).

Input Format:

Model fields

Output Format:

Generated Django model

Example 1 Input:

Student, name:char, age:int, email:email

Output:

```
class Student(models.Model):  
    name = models.CharField(max_length=100)  
    age = models.IntegerField()  
    email = models.EmailField()
```

Answer:

```
from django.db import models
```

```
class Student(models.Model):  
    name = models.CharField(max_length=100)  
    age = models.IntegerField()  
    email = models.EmailField()  
  
    def __str__(self):  
        return self.name
```

3. Django URL Mapping

Problem:

Write the urls.py entry to map the URL /students/ to the view student_list.

Input Format:

URL path and view function name

Output Format:

Correct URL mapping in Django

Example 1 Input:

/students/ student_list

Output:

```
path('students/', views.student_list, name='student_list')
```

```
from django.urls import path
```

```
from . import views
```

Answer:

```
from django.urls import path

from . import views
urlpatterns = [

    path('students/', views.student_list, name='student_list'),

]
```

in school_mgmt/urls.py:

```
from django.contrib import admin

from django.urls import path, include

urlpatterns = [

    path('admin/', admin.site.urls),

    path("", include('students.urls')),

]
```

4. Django View and Template

Problem:

Write a view `student_list` that passes a list of student names to the template `students.html`.

Input Format:

List of names

Output Format:

Render template with student names

Example 1 Input:

```
["Ali", "Rahul", "Sara"]
```

Output:

Template rendered with names: Ali, Rahul, Sara

Answer:

Django View:

in `students/views.py`:

```
from django.shortcuts import render
```

```
def student_list(request):  
    students = ["Ali", "Rahul", "Sara"]  
    return render(request, 'students.html', {'students': students})
```

Django template:

in students/templates/students.html:

```
<!DOCTYPE html>  
  
<html>  
  
<head>  
    <title>Students</title>  
</head>  
  
<body>  
    <h2>Student List</h2>  
    <ul>  
        {% for student in students %}  
            <li>{{ student }}</li>  
        {% endfor %}  
    </ul>  
</body>  
</html>
```

5. Django Form Validation

Problem:

Create a Django form for Students with validation ensuring age ≥ 18 .

Input Format:

Name, Age

Output Format:

Form validation message

Example 1 Input:

John, 16

Output:

ValidationError: Age must be at least 18

Example 2 Input:

Alice, 20

Output:

Form is valid

Answer:

in forms.py

```
from django import forms
```

```
from django.core.exceptions import ValidationError
```

```
class StudentForm(forms.Form):
```

```
    name = forms.CharField(max_length=100)
```

```
    age = forms.IntegerField()
```

```
    def clean_age(self):
```

```
        age = self.cleaned_data['age']
```

```
        if age < 18:
```

```
            raise ValidationError("Age must be at least 18")
```

```
        return age
```

6. Django Authentication

Problem:

Write a Django view for user login using `authenticate()` and `login()`.

Input Format:

Username, Password

Output Format:

Login success/failure

Example 1 Input:

(username=admin, password=wrongpass)

Output:

Invalid credentials

Example 2 Input:

(username=admin, password=admin123)

Output:

Login successful

Answer:

```
from django.contrib.auth import authenticate, login
```

```
from django.shortcuts import render
```

```
def user_login(request):
```

```
    if request.method == "POST":
```

```
        username = request.POST['username']
```

```
        password = request.POST['password']
```

```
        user = authenticate(request, username=username, password=password)
```

```
        if user is not None:
```

```
            login(request, user)
```

```
            return render(request, 'success.html', {"message": "Login successful"})
```

```
        else:
```

```
            return render(request, 'login.html', {"error": "Invalid credentials"})
```

```
    return render(request, 'login.html')
```

7. MySQL DDL Operation

Problem:

Write an SQL query to create a table courses with id, course_name, credits.

Input Format:

Table name and fields

Output Format:

DDL SQL query

Example Input:

courses id:int, course_name:varchar(100), credits:int

Output:

```
CREATE TABLE courses (  
    id INT PRIMARY KEY,  
    course_name VARCHAR(100),  
    credits INT  
);
```

Answer:

```
CREATE TABLE courses (  
    id INT PRIMARY KEY,  
    course_name VARCHAR(100),  
    credits INT  
);
```

8. MySQL DML Operation

Problem:

Write an SQL query to insert a new course record into the courses table.

Input Format:

Course details

Output Format:

SQL Insert statement

Example Input:

(1, "Mathematics", 4)

Output:

```
INSERT INTO courses VALUES (1, 'Mathematics', 4);
```

Answer:

```
INSERT INTO courses (id, course_name, credits)
```

```
VALUES (1, 'Mathematics', 4);
```

9. Flask Routing

Problem:

Create a Flask route `/hello/<name>` that returns `"Hello, <name>!"`.

Input Format:

Name in URL

Output Format:

Greeting string

Example 1 Input:

```
/hello/Ali
```

Output:

```
Hello, Ali!
```

Example 2 Input:

```
/hello/Sara
```

Output:

```
Hello, Sara!
```

Answer:

```
from flask import Flask
```

```
app = Flask(__name__)
```

```
@app.route('/hello/<name>')
```

```
def hello(name):
```

```
    return f"Hello, {name}!"
```

```
if __name__ == "__main__":  
    app.run(debug=True)
```

10. Flask Blueprint

Problem:

Create a Flask blueprint `students_bp` with a route `/students` returning "Student List Page".

Input Format:

Blueprint name and route

Output Format:

Blueprint route response

Example 1 Input:

`students_bp /students`

Output:

Student List Page

Answer:

students.py:

```
from flask import Blueprint  
  
students_bp = Blueprint('students_bp', __name__)  
  
@students_bp.route('/students')  
def student_list():  
    return "Student List Page"
```

app.py:

```
from flask import Flask  
  
from students import students_bp  
  
app = Flask(__name__)  
  
app.register_blueprint(students_bp)
```

```
if __name__ == "__main__":  
    app.run(debug=True)
```